



Prog C++

FS 2026 – Prof. Dr. Christian Werner

Autoren: Fabian Steiner

Mitwirkende: Mirko Ratti

1 Basics

1.1 Grundsätzlich

C++ ist sehr ähnlich zu C. Es gibt aber einige wichtige Änderungen zu C:

- zu benutzender Compiler heisst jetzt clang++
- Nativen Bool Typ : **bool**
- richtiges Konstanten Schlüsselwort : **constexpr**
- Standardbibliotheken haben kein .h mehr beim Aufrufen
- C Standartbibliotheken können weiter verwendet werden, haben aber ein C im Header(stdio.h → cstdio)
- Char-Literale (z. B. 'A') haben in C++ den Datentyp char (in C war es int)(Brauchen ein const)
- Es gibt benannte Namensräume. Wichtigster Namensraum für die STL: std

1.2 Hello world C++

```
1 #include <iostream> // iostream fuer Kommandozeilenausgabe
2 using namespace std; // dadurch entfaellt das "std::" vor cout
3 int main() // 'void' can be omitted
4 {
5     cout << "Hello World!" << endl; // "Einschieben" von hello
6     // world in den stream
7     return 0; // ein return 0 ist in c++ am Ende der main
8     funktion fakultativ
9 }
```

1.3 Range-based for Loop

Range based for Loops laufen automatisch ein Array ab, ohne das zuerst die Grösse ermittelt werden muss. Die C for loop syntax wurde etwas erweitert. Syntax:

```
1 for(<DataType> <IteratorName> : arrayName){
2     ;//Loop where <IteratorName> is each element
3 }
```

Siehe: Als Array Name wird ein Pointer auf das Array verlangt, nicht auf das erste Element! Ein foreach funktioniert **nicht** mit einem **Pointerpointer**.

1.4 Streams

Ein Stream repräsentiert einen generischen sequentiellen Datenstrom. Z.b: ein Eingabefeld, Dateien oder Netzwerktraffic.

Die wichtigsten Operatoren sind:

- << : Inserter → Daten einfügen
- >> : Extractor → Daten herausholen

Für Standardklassen sind diese Operatoren bereits definiert.

Standardstreams

- **cin** : Standard Eingabe
- **cout** : Standard Ausgabe
- **cerr** : Standard Fehlerausgabe,
- **clog** : Standard Log Ausgabe, möglicherweise mit cerr gekoppelt

cin/cout Immer ganz links!, operatoren klar und lesbar anordnen

1.4.1 Streamformatierung

Formatierungen der Streams können mit folgenden Schlüsselwörtern erreicht werden:

Flag	Wirkung	Gültigkeit
boolalpha	bool-Werte werden textuell ausgegeben	Dauerhaft
dec	Ausgabe erfolgt dezimal	Dauerhaft
fixed	Gleitkommazahlen im Fixpunktformat	Dauerhaft
hex	Ausgabe erfolgt hexadezimal	Dauerhaft
internal	Ausgabe innerhalb Feld	Dauerhaft
left	linksbündig	Dauerhaft
oct	Ausgabe erfolgt oktal	Dauerhaft
right	rechtsbündig	Dauerhaft
scientific	Gleitkommazahl wissenschaftlich	Dauerhaft
showbase	Zahlenbasis wird gezeigt	Dauerhaft
showpoint	Dezimalpunkt wird immer ausgegeben	Dauerhaft
showpos	Vorzeichen bei positiven Zahlen anzeigen	Dauerhaft
skipws	Führende Whitespaces ignorieren	Dauerhaft
unitbuf	Leert Buffer dopo ogni operazione	Dauerhaft
uppercase	Kleinbuchstaben in Grossbuchstaben wandeln	Dauerhaft
<i>Benötigen Header <iomanip></i>		
setw(n)	Setzt minimale Feldbreite	Einmalig
setprecision(n)	Setzt Anzahl Stellen (signifikant o. Nachkomma)	Dauerhaft
setfill(c)	Setzt Füllzeichen (Standard: Leerzeichen)	Dauerhaft

1.5 Namespaces

- Namespaces helfen, Namenskonflikte zu verhindern. Sie fügen ein Namensvorsatz zu allen Variablen.
- Per ogni Unit (Modul) dovrebbe essere definito un namespace
- **Mit kleine Buchstaben starten**
- namespaces senza nome: evitano uso di static per var. locali
- L'operatore :: ritorna sempre la variabile o funz. più globale

```
1 void f(); // name in global Namespace (1. f())
2 namespace myLib{void f(); int i} // namespace myLib (2. f())
3 f(); // f von global Namespace
4 myLib::f(); // f von namespace myLib
5 using namespace myLib;
6 using myLib::i; // posso usare using anche per singole var.
7 ::f(); // 1. f()
8 f(); // 2. f()
9
10 // Nameless namespace
11 namespace {
12     int localCounter = 0; // Invisible all'esterno di questo
13     file
14 }
15 void funzionePubblica() {
16     localCounter++; // instead of static
17 }
```

1.6 Kostanten

- #define soll nich mehr verwendet werden
- constexpr verwenden!
- enum da usare solo se l'automatismo di +1 è utile

1.6.1 Optiemierung constexpr

Ambito	Sintassi Consigliata
Header file	inline constexpr
Class or function	static constexpr

2 Funktionen

2.1 Default Argumente

- **Prototipo**: I valori predefiniti si assegnano **solo** nell'interfaccia.
- **Omissione**: I parametri con default possono essere saltati nella chiamata.
- **Ordine**: I parametri di default devono trovarsi **sempre alla fine (a destra)** della lista parametri.

```
1 void func(int a = 0, int b = 0, int c = 0){}
2 func(); // a = 0, b = 0, c = 0
3 func(1); // a = 1, b = 0, c = 0
4 func(1, 1); // a = 1, b = 1, c = 0
5 func(1, 1, 1); // a = 1, b = 1, c = 1
6 //Alle Funktionsaufrufe valide
7
8 void func2(int a, int b = 0, int c = 0); // korrekte
9 Reihenfolge
```

Default Argumente können überall verwendet werden bis auf Aufrufparameter von Operatoroverloading. Dort sind sie **nicht** erlaubt. Dort machen sie aber ohnehin keinen Sinn.

2.2 Funzioni Inline

- **Sostituto Macro**: Alternativa sicura alle macro C.
- **Zero Overhead**: Codice inserito direttamente (nessun salto di funzione).
- **Type Safety**: Il compilatore garantisce il controllo dei tipi.
- **Target**: Funzioni piccole chiamate frequentemente (es. cicli).
- **No-go**: Inapplicabile a ricorsione o puntatori a funzione.
- **Header-only**: Se usato nei file Header, definizione func. direttamente in file.h
- **Ottimizzazione**: Richiede flag attivi (es. -O3).

```
1 // Definizione nell'Header (.h)
2 inline int quadrato(int x) { return x * x; }
3
4 // Compilazione con ottimizzazione
5 clang++ -O3 main.cpp
```

3 Objektorientierung

Objektorientierung soll es erleichtern, eine Abbildung der Realität zu erstellen. Viele Dinge aus der Wirklichkeit können als Modell dargestellt / simplifiziert werden. Diese Modelle können in Software in sogenannte Objekte umgewandelt werden.

3.1 Objekte

Objekte stellen Dinge, Sachverhalte oder Prozesse dar. Sie sind ein rein gedankliches Konzept.

Sie kennzeichnen sich durch:

- eine **Identität**, welche es erlaubt Objekte voneinander zu unterscheiden
- **statische Eigenschaften** zur Darstellung des **Zustandes** des Objekts in Form von **Attributen**
- **dynamische Eigenschaften** zur Darstellung des **Verhaltens** des Objekts in Form von **Methoden**

3.2 Klassen & Instanzen

Ähnliche Objekte können in **Klassen** zusammengefasst werden, um Programmieraufwand tiefer zu halten. Verwendete Objekte werden als **Instanz** eingesetzt. Diese Objekte haben dieselben Attribute, allerdings andere Werte. Eine Beispielklasse:

```
1 #include <string> //Stl Strings
2 class Student {
3     public: // folgende Elemente sind Public
4         int IDNumber;
5         void doStuff(int time = 0);
6     private: // folgende Elemente sind Private(standart)
7         std::string name; //String
8         float bierkonsum;
9         unsigned long long int investedHoursInET;
10 };
11 // ----- start cpp -----
12 //implementierung der doStuff Funktion
13 void Student::doStuff(int time = 0) {
14 } //ToDo
15
16 int main(){
17     Student typETStudent; //instanz erzeugen
18     typETStudent.IDNumber = 1; //Attribute veraendern
19     typETStudent.doStuff(); //Methode rufen
20 }
```

3.2.1 Reihenfolge der Header-Datei (.h)

1. Dateikommentar mit Lizenzvereinbarung.
2. *Includes des verwendete System Header <...>*
3. *Includes der Projektbezogenen Header "..."*
4. *Definition von Konstanten*
5. *Typedef's und Definitionen von Strukturen*
6. *allenfalls extern-Deklaration von Global-Variablen*
7. *Funktionsprototypen, ink. Kommentar der Schnittstelle, bzw. Klassendeklarationen*

- Die *blauen* Punkte müssen sich im Includeguard befinden(#pragma once).
- Pro Header-Datei sollte nur eine Oberklasse deklariert sein.
- Kleine Unterklassen können im gleichen Header stehen.
- Kein using namespace ... in header files

3.2.2 Reihenfolge der Implementation (.cpp)

1. Dateikommentar mit Lizenzvereinbarung
2. includes der eigenen Header
3. includes der Projektbezogenen Header
4. includes der verwendeten System Header
5. allenfalls globale und statische Variablen
6. Präprozessor-Direktiven
7. Funktionsprototypen von lokalen, internen Funktionen (in nameless Namespace)
8. Definition von Funktionen und Klassen (**Kommentare nicht wiederholen**)

3.3 UML

Die Unified Modeling Language ist eine normierte Sprache um Objekte grafisch darzustellen, **programmierspracheunabhängig**.

3.3.1 UML-Klassendiagramm Notation

Besteht immer aus 3 Boxen:

Student
+ IDNumber : int # name : string - bierkonsum : float # investedTimeInET : int
- doStuff(time : int) + useless()

- + : public (überall sichtbar)
- - : private (nur in aktueller Klasse sichtbar)
- # : protected (in aktueller und Unterklassen sichtbar)
- *Italic* : Funktion oder Klasse ist Abstrakt (=0)
- Unterstrichen : Funktion oder Attribut ist static
- <Name der Klasse> : Konstruktor (ohne <>)
- ~<Name der Klasse> : Destruktor (ohne <>)

3.4 class vs struct

Eine class und struct können fast identisch verwendet werden. Eine class hat **standardmässig Sichtbarkeit private** sonst muss explizit mit public: gekennzeichnet, Struct public(wenn nichts definiert wird).

3.4.1 Zugriffsschutz

Modifikatore	Beschreibung e Best Practice
public	Accessiblen dall'interno e dall'esterno della classe. → Quasi tutti i metodi sono pubblici. → attributi non dovrebbero <i>mai</i> essere pubblici.
protected	Accessiblen dall'interno della classe e dalle classi derivate . → Da utilizzare con parsimonia.
private	Accessiblen solo all'interno della classe stessa. → Per tutti gli attributi e per metodi locali specifici.

3.5 Inkludierte Dokumentation

Mann kann im H-File direkt die Dokumentation zu der Schnittstelle schreiben. Das ermöglicht das einfachere Verwenden der Schnittstelle.

```
1 /**
2  * @brief Kurzzusammenfassung der Klasse
3  *
4  */
5 class stuff{
6     public:
7         /**
8          * @brief Funktion von x
9          *
10         */
11         double x;
12         /**
13          *
14          * @brief Kurzzusammenfassung der Funktion
15          *
16          * @param eingang Funktion des Parameter
17          * @return nix, aber falls...
18          *
19          */
20         void doStuff(stuff eingang);
21 };
```

- Zu oberst der Name
- In der Mitte Attribute(Variablen)
- Unten Methoden.
- Methoden und Attribute fangen mit einem Symbol an, welches die **Sichtbarkeit** definiert und haben ein bestimmtes Merkmal:
- Parole come void o const sono specifiche per linguaggio, da non includere

Es kann immer mehr geschrieben werden, man sollte sich aber auf das Nötige beschränken.

4 Konstruktoren und Destruktoren

I costruttori (CTOR) e i distruttori (DTOR) sono metodi speciali che gestiscono il ciclo di vita di un oggetto. Entrambi hanno lo stesso nome della classe e **non** hanno alcun tipo di ritorno (nemmeno void).

4.1 Konstruktoren (CTOR)

Il compito principale del CTOR è l'inizializzazione "pulita" di tutti gli attributi della classe.

- **Default-Konstruktor:** Non ha parametri. Viene invocato automaticamente se l'oggetto è dichiarato senza argomenti (es. Stack s;). Se non definito, il compilatore ne crea uno implicito (che però non inizializza i tipi primitivi).
- **Copy-Konstruktor:** Riceve una const-Referenz alla propria classe (TString(const TString& s)). Viene usato per creare un oggetto come copia di uno esistente.
- **Explicit-Konstruktor:** Marcando un CTOR a parametro singolo con explicit, si impediscono conversioni implicite indesiderate (es. TString s = 5;).

Werden verschiedene Konstruktoren definiert, muss der Default zuerst aufgerufen werden, wenn dieser erhalten bleiben soll.

4.1.1 Best Practices di Inizializzazione

Una delle responsabilità primarie del costruttore è garantire che l'oggetto nasca in uno stato valido. Ciò implica che **tutti** gli attributi della classe debbano essere inizializzati su un valore definito.

- **Assegnamento nel corpo (4.4):** Meno efficiente, i membri vengono prima creati con valori predefiniti e poi sovrascritti.
- **Inizialisierungsliste (5.3):** Il metodo preferito per motivi di efficienza; gli attributi vengono inizializzati direttamente al momento della creazione.
- **Member In-Class Initialization:** (Da C++11) Permette di impostare valori di default direttamente nella dichiarazione della classe, semplificando i costruttori ed evitando dimenticanze.

4.2 Gestione Memoria: Shallow vs. Deep Copy

Quando una classe gestisce puntatori a memoria allocata sul Heap (new), il comportamento di default è critico:

Shallow Copy (Default): Il compilatore copia solo l'indirizzo del puntatore. Entrambi gli oggetti puntano alla stessa memoria. Se uno la libera, l'altro punta a memoria non valida (*Double Free* o *Segmentation Fault*).

Deep Copy (Manuale): **È necessario** definire un Copy-Konstruktor proprio che alloci nuovo spazio sul Heap e copi i dati effettivi.

4.3 Destruktor

Ein Destruktor entfernt eine Instanz aus dem Speicher, hat keine Aufrufparameter und auch kein Rückgabewert. Es kann immer nur **einen** Destruktor geben. Wenn der Destruktor fertig ist, sollte kein Speicher mehr vom Objekt belegt sein.

Destruktoren sollten immer **virtuell definiert werden** (siehe 10.2.1). Mit anderen virtuellen Funktionen **müssen** sie virtuel definiert werden. Der Funktionsname beginnt mit einer Tilde(~). Der Destruktor wird evtl. auch Implizit aufgerufen z.b., wenn aus einer Funktion wieder herausgesprungen wird.

4.4 Beispiel

```
1 // START H FILE
2 class Storage{
3     public:
4         Storage();
5         virtual ~Storage();
6         void add(int in);
```

```
7     void nix(Storage inC);
8     protected:
9         ...
10    private:
11        int* data;
12        int size;
13 };//eine Klasse die Int Werte speichert, aehnlich wie ein
    array
14
15 // END H FILE
16 // START Storage.cpp
17 Storage::Storage() { // Konstruktor
18     data = nullptr;
19     size = 0;
20 }
21
22 Storage::~Storage() { // Destruktor
23     delete[] data;
24     data = nullptr;
25     size = 0; // Speicher der allokiert wurde sollte hier
        freigegeben werden
26 }
27 void Storage::add(int in){
28     // todo
29 }
30 void Storage::nix(Storage inC){
31     // todo
32 }
33
34 // END Storage.cpp
35 // START main.cpp
36
37 int main(){
38
39     Storage* i1 = new Storage; // default konstruktor
40     Storage i2; // default konstruktor
41     i1->nix(i2); // Call-by-Value. Eine Kopie von i2 wird auf
        den Stack gelegt -> Copy-Konstruktor!
42
43     delete i1; // destruktur von i1 wird explizit aufgerufen
44     return; // destruktur von i2 wird implizit aufgerufen
45 }
```

5 Initialisierungslisten und direkte Initialisierung

Die Initialisierung von Datentypen kann auf verschiedene Arten erreicht werden. Es wird zwischen Zuweisung und Initialisierung unterschieden.

5.1 POD's

plain old datastructure / built in datatypes

```
1 // Zuweisung fuer POD:
2 int i; // Speicher fuer Instanz wird alloziert, Wert ist
    uninitialisiert
3 i = 5; // Wert (aus anderer Speicherstelle) wird zugewiesen
4
5 // Initialisierung fuer POD: Kein weiterer Speicherverbrauch
6 int i = 5; // (C-Style)
7 int i(5); // (C++-Style bis C++11)
8 int i{5}; // (neuer C++-Style)
```

```
9 // Aus Effizienzgrunden bevorzugen wir (fast) immer die (
    direkte) Initialisierung!
```

5.2 Non PODs

```
1 Aclass m;//kein Initialwert
2 // Vorsicht: Falls "Aclass" gross ist, muss viel kopiert
    werden
3 m = otherInstance;
4
5 // Copy-Initialisierung fuer non-POD:
6 Aclass m = otherInstance; //(copy ctor)
7 Aclass m(otherInstance); // C++-Schreibweise
8 Aclass m{otherInstance}; // C++-Schreibweise seit c++11
9
10 //Besser: direkte Initialisierung auch fuer non-PODs:
11 Aclass m("Eagle 1", 1, 2, ...); // (vor C++11)
12 Aclass m{"Eagle 1", 1, 2, ...}; // bevorzugte Schreibw. seit C
    ++11
13 // Die Instanz wird ohne Umwege mit den gewuenschten Werten in
    den Speicher geschrieben, sofern ein geeigneter user-
    defined ctor vorhanden ist!
```

5.3 User defined Constructor

Um eine Klasse richtig initialisieren zu können muss der Ctor korrekt definiert werden. Eine sogenannte Initialisierungsliste wird verwendet:

```
1 student::student(int _semester, long int _eltVerzweiflung):
2     semester{_semester},
3     eltVerzweiflung{_eltVerzweiflung}
4 //die Reihenfolge der Initialisierung ist durch die
    Reihenfolge im Speicher gegeben!
5 {
6     //Rumpf kann leer bleiben Initialisierung bereits fertig
7 }
```

6 Assertions

Assertions sollten Annahmen über korrekte Wertebelegung darstellen. Sie sind kein C spezifisches Konzept und sollten nur zu Testzwecken eingesetzt werden. Im Releasebuild sollten sie deaktiviert werden. Dafür gibt es spezifische Präprozessorflags. Der Header <assert.h> bzw. <cassert> stellt hierfür ein Präprozessor-Makro assert (Bedingung) zur Verfügung, um solche Zusicherungen auszudrücken.

Beispiel:

```
1 // #define NDEBUG // Deaktiviert Assertions im Projekt
2 #include <cassert>
3 bool testStuff(int* in1){
4
5     assert(in1 != nullptr);
6     //assertion
7
8     ;//restliche implementation einer Funktion
9
10 }
```

Das obige Beispiel hat eine Assertion. Diese würde im Test, falls ein Nullpointer übergeben wird ein Fehler ausgegeben.

7 This-Pointer

Mit dem this Pointer kann ein Pointer des aufrufenden Objekt zurückgegeben werden. Bsp:

```
1 class Stuff{
2     public:
3         Stuff& funktion(int valIn);
4     private:
5         int value;
6 };
7 Stuff& Stuff::funktion(int valIn){
8     //do Stuff with val z.b. addieren auf interne Variabel
9     this->funktion(0); // auch moeglich
10    return *this; //this pointer -> wird dereferenziert und
        Referenz zurueckgegeben
11 }
12 int main(){
13     Stuff memb;
14     memb.funktion(1).funktion(2).funktion(3);
15 }
```

Nacheinander werden diese Funktionen durchlaufen da Funktion(2) vom zurückgegebenen Pointer aus Funktionsaufruf 1 aus gerufen wird. Die funktion wird Kaskadiert

8 Overloading

Operator Overloading bedeutet das bestehende Operatoren überladen werden im Sinne neu definiert / darüber geladen. Sie erhalten neue Bedeutung für eine Klasse.

8.1 Methodenoverloading

- **Definizione:** Stesso nome per funzioni con parametri diversi (tipo, numero o ordine).
- **Firma:** Composta da *Nome* + *Parametri*. Il tipo di ritorno è **escluso**.
- **Regola:** Se differiscono solo per il tipo di ritorno, il compilatore genera un errore.
- **Best Practice:** Usare l'overloading solo per operazioni comparabili e **non mescolarlo mai con i parametri di default**.

Caratteristica	Overloading	Parametri Default
Memoria	Più funzioni	Singola funzione
Manutenzione	Maggiore	Minore
Flessibilità	Tipi diversi	Valori predefiniti

```
1 // Approccio Overloading (3 implementazioni)
2 void print(int i);
3 void print(char c);
4 void print(int* arr, int n);
5 // A dipendena del parametro passato verra' chiamata una div.
    implementazione di print()
```

8.2 Operator overloading

Es können die inkludierten Operatoren von Cpp überladen werden. Erlaubte Operatoren sind:

+	-	*	/	%	^	&		~
!	=	<	>	+=	-=	*=	/=	%=
^=	&=	=	<<	>>	<<=	>>=	==	!=
<=	>=	&&		++	--	,	->*	->
()	[]	new	delete	new[]	delete[]			

Nicht erlaubte Operatoren sind: . .* :: und ?.:.

Es wird zwischen overloading global und in Methode unterschieden.

8.2.1 Global

Globales overloading funktioniert mit einer als friend deklarierten Funktion. Als friend deklarierte Funktionen agieren als **globale Funktion** und haben vollen zugriff auf **alle Parameter einer Klasse**. Sie sind daher zu vermeiden da sie eher ein Workaround sind und zu Problemen führen können.

Syntax : <returntype> operator<type>(type val1, type val2);
BSP:

```
1 // H file
2 class Dozent{
3     friend Dozent& operator+(Dozent& in1, Dozent& in2);
4         //der + Operator wird ueberschrieben
5     private:
6         int numb;
7 };
8
9 Dozent& operator+(Dozent& in1, Dozent& in2);
10
11 // ----- start cpp -----
12 Dozent& operator+(Dozent& in1, Dozent& in2){
13     in1.numb += in2.numb; // whatever Operation welche zum
14     + Operator passt
15     return in1;
16 }
```

Die Funktion kann von überall aus dem Programm ausgeführt werden. Es sollte nur dann verwendet werden, wenn der erste Parameter (links) nicht eine Instanz der Klasse ist da der erste Aufrufparameter immer die Instanz sein muss.

8.2.2 Methoden

Ein Operator kann auch als Methode einer Klasse definiert werden. Sie haben nur noch ein Aufrufparameter.
Syntax : <returntype> operator<typ>(type in);
Der 2. Parameter ist das Objekt selbst, welche die Methode aufruft.

```
1 // H file
2 class Dozent{
3     friend Dozent& operator+(int in1, Dozent& in2);
4     public: // friends immer vors public
5         Dozent& operator+(Dozent& in1); // der + Operator wird
6         ueberschrieben (Falls eine andere Instanz addiert wird)
7         Dozent& operator+(int in1); // der + Operator wird
8         ueberschrieben (falls ein int addiert wird)
9     private:
10         int number;
11 };
12
13 Dozent& operator+(int in1, Dozent& in2); // dieser Fall muss
14     global / mit friend geloest werden
15
16 // ----- start cpp -----
17 Dozent& operator+(int in1, Dozent& in2){
18     in2.number += in1;
19     return in2;
20 }
21
22 Dozent& Dozent::operator+(Dozent& in2){
```

```
22     number += in2;
23     return *this;
24 }
25
26 // ----- start main file -----
27 main(){
28     Dozent D1;
29     Dozent D2;
30     D1 + D2; // fall mit anderer Instanz
31     D1 + 1; // fall mit int
32     1 + D1; // durch Global overloading abgedeckt
33 }
```

Die obigen Funktionen machen alle dasselbe. Sie unterscheiden sich durch die **unterschiedlichen Argumente**. Diese decken verschiedene Eingabetypen ab. Es wäre schlechter Stil, die Typkonvertierung dem Compiler implizit zu überlassen! Ein Fall, bei dem das anderstypige Argument zuerst kommt, muss noch per Globalem Overloading gelöst werden. Da der erste Aufrufparameter, beim Overloading mit Methoden, immer zuerst kommt.

9 Getter & Settermethoden

Getter und Settermethoden werden Typischerweise verwendet um Lese und Schreibzugriffe auf eine Klasse zu regeln. Attribute sind **alle Parameter einer Klasse**. Mit einer Get oder Set Methode kann der Zugriff auf diese kontrolliert / angepasst werden.

const after the funtion name declares that the method cannot modify any Attribute
immer fur Selector (Getter) benutzen

```
1 #include <string>
2 class Stuff{
3     public:
4         const std::string& getName() const;
5     protected: // erlaubt schreibenden Zugriff von Unterklassen
6         void setName(const std::string& in);
7     private:
8         std::string name; // privates Attribut
9 };
10 const std::string& Stuff::getName() const { // Getter
11     return name;
12 }
13 void Stuff::setName(const std::string& in) { // Setter
14     name = in;
15 }
```

9.0.1 Mutable

Si può permettere a determinati attributi di essere modificati da metodi const con la keyword mutable, usare il meno possibile

```
1 ...
2 mutable int error;
3 ...
```

9.0.2 Const correctness

- **Sola Lettura:** Un oggetto dichiarato come const può invocare *solo* metodi dichiarati const.
- **Sicurezza:** Se un metodo non è marcato const, il compilatore assume che possa modificare l'oggetto e ne impedisce l'uso su istanze costanti per evitare effetti collaterali indesiderati.

Beispiel:

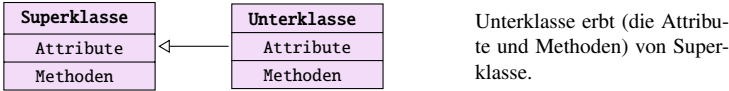
```
1 class Stack {
2     public:
3         int pop(); // Modifica lo stato (rimuove
4         bool isEmpty() const; // Sola lettura (osserva lo stato
5         );
6
7     void check(const Stack& s) {
8         s.isEmpty(); // OK: il metodo promette di non modificare
9         s
10        // s.pop(); // ERRORE: pop() potrebbe modificare s
11    }
```

10 Vererbung

Vererbung erlaubt es Attribute und Methoden von anderen Klassen zu übernehmen. Die Oberklasse, Baissklasse oder Superklasse **vererbt** an eine Unterklasse, derived class oder eine Spezialisierung. Bei der Vererbung werden immer **alle** Attribute oder Methoden weitergegeben.

10.0.1 UML

In einem UML Diagramm wird vererbung wie folgt dargestellt:

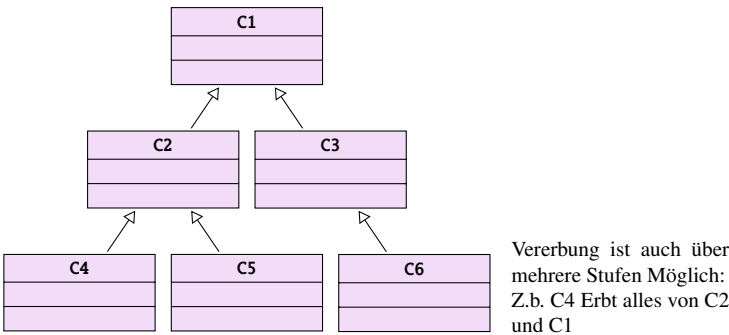


Dies stellt eine **ist ein** beziehung dar. Es ist sehr wichtig, dass die Pfeilspitze **genau so!** aussieht wie in der Darstellung da ansonsten Verwechslungsgefahr mit anderen Mechanismen entstehen könnte.

10.0.2 Syntax

```
1 class Unterklasse : public Superklasse{
2     ...
3 } ; // Unterklasse erbt Superklasse
```

10.0.3 UML++



10.0.4 Sichtbarkeit

Die Sichtbarkeit ist durch das Public definiert. Es ist möglich die Sichtbarkeit aller Attribute und Methoden einzuschränken. Mit **Public** wird alles mit der originalen Sichtbarkeit übernommen. Mit **protected** wird alles was public war protected. Mit **private** wird alles private (nicht sehr nützlich).

Inheritance	In base class	In derived class
public (normal case)	public protected private	public protected private
protected	public protected private	public protected private
private	public protected private	public protected private

Wird eine Vererbung private durchgeführt, sind keine Attribute oder Methoden mehr sichtbar.

Attribute soll nie protected sein

10.0.5 Constructor / Destructor chaining

Logica dei Costruttori (CTOR)

- **Principio di Responsabilità:** Il costruttore di una classe inizializza esplicitamente **solo i propri attributi**.
- **Delegazione:** Per gli attributi della classe base, viene invocato il costruttore della classe base stessa (la base 'si occupa delle proprie faccende').
- **Regola di Priorità:** Gli elementi della classe base devono essere **sempre iniziati per primi**.

```

1 Unterklasse::unterklasse(int _initvalOberklasse,
2                             int _initvallUnterklasse):
3     Oberklasse{_initvalOberklasse},// muss zuerst stehen da
4     // diese zuerst initialisiert werden muss
5     unterklassenAttribut{_initvallUnterklasse}
6 {} // bleibt leer

```

Logica dei Distruttori (DTOR)

- **Ordine Inverso:** I distruttori vengono eseguiti nell’ordine esattamente opposto ai costruttori.
- **Ordine di Esecuzione:** Dal basso verso l’alto (Sottoclasse → Classe Base).

10.1 Überschreiben von Methoden

Geerbte Funktionen können überschrieben werden. Die Unterklasse definiert eine neue Methode mit demselben Namen. Allerdings kann es dann zu Mehrdeutigkeit kommen:

Im folgenden Beispiel haben eine Subklasse und eine Oberklasse eine print Funktion welche **nicht** virtual gekennzeichnet ist:

```

1 int main() {
2     Subclass p;           // Klassenerstellung
3     p.print();            // print von Subclass, ok
4     Subclass* sPtr = &p; // Subclass Pointer
5     sPtr->print();         // print von Subclass, ok
6     Topclass* tPtr = &p;  // !! Topclass Pointer !!
7     tPtr->print();         // print von Topclass!!!!!!
8     Subclass& sRef = p;   // Subclass ref
9     sRef.print();         // print von Subclass, ok
10    Topclass& tRef = p;   // !! Topclass ref !!
11    tRef.print();         // print von Topclass!!!!!!
12 }

```

Dieses Beispiel soll zeigen, dass Pointer, welche eigentlich von einer Oberklasse auf eine Unterklasse zeigen auf die Eigenen Funktionen der Oberklasse zeigt, solange diese nicht überschrieben worden sind. Dies kann als Verhalten gewünscht sein.

10.2 virtual, final, override und scopeoperator

10.2.1 virtual

- **Zweck:** Kennzeichnet Methoden, die in Unterklassen überschrieben werden sollen. Dies ermöglicht **Dynamic Binding**, sodass zur Laufzeit immer die spezialisierte Methode der Unterklasse aufgerufen wird.
- **Polymorphie:** Eine Klasse, die mindestens eine virtuelle Funktion besitzt, wird als **polymorph** bezeichnet.
- **Vererbung:** Einmal als virtual deklariert, bleibt die Methode in der gesamten Vererbungskette virtuell.
- **Wichtige Syntax-Regel:** Das Schlüsselwort virtual darf **nur bei der Deklaration** (im H-File) verwendet werden.

10.2.2 Virtual beim Dtor

Ist eine Methode als virtual deklariert, so muss der Dtor **ZWINGEND** auch als virtual deklariert werden.

10.2.3 Override

Soll eine Methode eine geerbte Methode ersetzen so muss diese mit Override gekennzeichnet werden. Solch eine Methode ist automatisch auch virtual. Der Compiler kann so überprüfen, ob eine virtual Methode vorhanden ist.

10.2.4 Final

Final kann zum einen verbieten, dass eine Methode einer Klasse überschrieben wird (Logischerweise geht das nur bei virtuellen Methoden). Zum anderen kann sie auch das weitere Erben einer Klasse verbieten.

10.2.5 Scopeoperator

Per Scopeoperator kann (::) eine Methode aus einer Oberklasse gezielt aufgerufen werden. Mit diesem wird garantiert die Funktion welche definiert wird aufgerufen, egal ob diese aufgrund virtual überschrieben wird.

```

1 class Subexample : public Upclass{
2     virtual ~Subexample(); // Correct Dtor
3     virtual void newMethod(); //gets overridden when inherited
4     // void oldVirtualMethod(); this inherited func doesnt get
5     // changed, it stays virtual
6     void getsOverridden() override;// overrides method in
7     // upclass
8     virtual void iAmPerfect() final;// cant be overridden
9 };
10 void Subexample::newMethod(){
11     Upclass::aMethodfromUpclass();
12     //calls a class from a class higher in the chain
13 }

```

10.3 Classi Polimorfe e Binding

- **Definizione:** Una classe è polimorfa se dichiara almeno una funzione virtual.
- **Tipi di Dato:**
 - **Statischer Datentyp (Tipo Statico):** Il tipo definito alla dichiarazione (tipo del puntatore o della referenza).
 - **Dinamischer Datentyp (Tipo Dinamico):** Il tipo effettivo dell’oggetto a runtime (l’istanza reale).
- **Static Binding (Early Binding):** Avviene durante la compilazione. È lo standard per funzioni non-virtuali e non comporta overhead.

- **Dynamic Binding (Late Binding):** La scelta della funzione avviene a runtime in base al tipo dinamico. Funziona solo tramite puntatori o referenze e comporta un leggero overhead gestito tramite V-Table.

10.3.1 Best Practice e Regole

- **Distruttori:** Devono essere obbligatoriamente virtual nelle classi base per garantire la corretta deallocazione delle sottoclassi, specialmente se gestite tramite puntatori alla classe base.
- **Costruttori:** Non possono mai essere dichiarati virtual.
- **Metodi:** Dichiarare virtual solo se è prevista la sovrascrittura. I metodi non-virtuali non devono essere sovrascritti nelle classi derivate.
- **Override:** Nelle sottoclassi, i metodi sovrascritti devono essere contrassegnati esplicitamente con la parola chiave override.

10.3.2 RTTI (Run-time Type Information)

L’RTTI permette di identificare il tipo di un oggetto durante l’esecuzione. Si utilizza la funzione typeid() inclusa nell’header <typeid>, che restituisce un oggetto di tipo std::type_info.

```

1 #include <typeid>
2 int main() {
3     int a{0};
4     float b{0.0f};
5     if (typeid(a) != typeid(b)) { // is always true
6         // because they have a different type
7     }
8 }

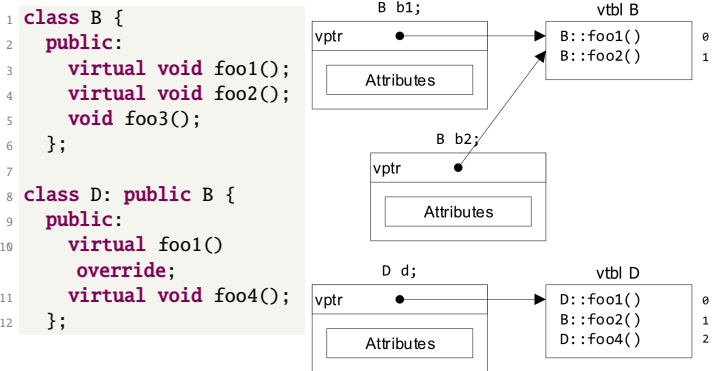
```

10.3.3 Implementazione: La V-Table

L’implementazione tecnica del polimorfismo e del RTTI si basa sulla **V-Table**:

- Ogni classe polimorfa possiede una V-Table contenente i puntatori alle sue funzioni virtuali.
- Ogni istanza della classe contiene un puntatore nascosto (**V-pointer**) che punta alla V-Table della propria classe.
- A runtime, l’istanza utilizza il V-pointer per risolvere l’indirizzo della funzione corretta.

Se un array di puntatori alla classe base contiene oggetti di diverse sottoclassi, ogni oggetto avrà un V-pointer che punta alla V-Table specifica della propria classe, garantendo che vengano chiamate le versioni corrette dei metodi sovrascritti.



10.4 Abstrakte Klassen

Wenn Klassen zwar festlegt, dass eine Methode da ist, diese aber noch nicht implementieren, nennt man das eine abstrakte Methode. Man spricht dann von einer abstrakten Klasse. Eine abstrakte Methode wird in UML *kursiv* dargestellt. Es ist egal, ob die abstrakten Methoden selbst definiert sind oder geerbt. Abstrakte Klassen kann

man nicht instanziierten. Es gibt kein spezielles Schlüsselwort dafür, man weist der Methode bei Definition 0 zu:

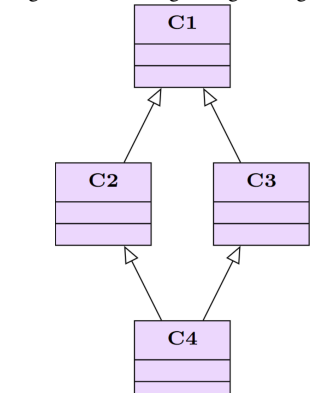
```
1 class ModernArt{// abstrakte klasse (wegen abstrakter Methode)
2     virtual void aMethod() = 0;// abstrakte Methode
3 };
```

10.5 Organisation der H files

Generell gilt immer noch: Jedes H File hat ein Cpp File. Allerdings können Unterklassen in ein H File zusammengefasst werden, falls diese nicht allzu Umfänglich sind. Ansonsten sollte ein neues H File erstellt werden. Für abstrakte Klassen fällt die Implementierung in einem Cpp file weg.

10.6 Mehrfachvererbung

Folgender Vererbungszweig ist möglich:



Solche Gebilde sind zwar möglich, sollten jedoch vermieden werden, da schnell sehr kompliziert und kaum lesbar. Es entsteht schnell Mehrdeutigkeit durch die verschiedenen Erbzweige. Bei der Definition einer Klasse können weitere Oberklassen mit einem Komma getrennt angegeben werden :

```
1 class Unterklasse : public Oberklasse1, public Oberklasse2{;
```

10.7 Static

Static im Zusammenhang mit Klassen bindet eine Methode oder ein Attribut an die Objektklasse und nicht an die Objektinstanz (ähnlich zu Python Klassenvariablen).

10.7.1 Methoden

Static Methoden müssen im Header-File deklariert und im Cpp-File definiert werden. Sie dürfen nur auf Static definierte Attribute zugreifen da **keine Objektinstanz vorhanden**. Hauptanwendungen sind Hilfsfunktionen wie Funktionsaufrufcounter oder Umrechnungsfunktionen. Bei der Deklaration wird der Klassennamen verwendet (siehe Bsp).

10.7.2 Attribute

Static Attribute müssen im H deklariert **und im Cpp File** definiert werden. Sie werden verwendet für z.B. Objektspezifische Konstanten oder im Zusammenhang mit Static Counter. (class scope)

10.7.3 Bsp

```
1 // Logger.h
2 class Logger {
3     public:
4         static void log(const std::string& message);
5     private:
6         static int nrObjects; // Non inicializzabile dirett.
7 };
8 // Logger.cpp
9 int Logger::nrObjects = 34; // Necesario!
```

```
10 // main.cpp
11 int main() {
12     // Benutze log aus Logger, ohne Instanz:
13     Logger::log("C++ ist super!");// class name wird verwendet
14     !
15     return 0;
16 }
```

11 Memorymanagement

Es gibt verschiedene Gründe warum nicht immer gleich viel Speicher verwendet wird z.B. da nicht unbegrenzt vorhanden ist. Speicher wird nur dann verwendet, wenn er wirklich gebraucht wird. Das Memorymanagement muss aktiv beachtet werden. Es wird zwischen automatischen und manuellen Speichermanagement unterschieden.

11.1 Automatisch

Automatisches Speichermanagement wird anhand der Lebenszeit einer Variable festgestellt und während dem Programmieren bereits festgelegt. Der Speicher liegt auf dem sogenannten **Stapel / Stack**.

Wird eine Funktion aufgerufen, werden die benötigten Variablen auf dem Stack definiert. Ebenso wird eine Returnadresse zur Funktion, die die Funktion aufgerufen hat abgelegt. Wie die Daten angeordnet werden, ist Architektur-abhängig. Wird die Funktion wieder verlassen, werden die Variablen vom Stack wieder entfernt und zur aufrufenden Funktion zurückgesprungen.

11.2 Manuell/dynamisch

Man kann auch manuell Speicher anfordern. Zum Beispiel, falls zur Compilezeit nicht bekannt ist, wie viel Speicher gebraucht wird. Dieser Speicher ist auf dem sogenannten **Haufen / Heap**.

Der Heap ist ein Speicherbereich, der vom Betriebssystem bereitgestellt und verwaltet wird.

Zur Laufzeit wird Speicher dynamisch angefordert. Dieser bleibt so lange belegt bis dieser Manuell wieder freigegeben wird.

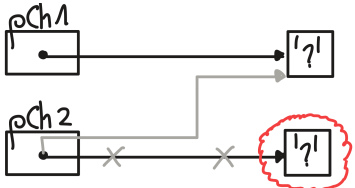
Es kann aber auch sein, falls der Speicher stark fragmentiert ist, das kein Speicher bereitgestellt werden kann. Generell ist bei sehr vollem Speicher das Arbeiten erschwert.

11.2.1 Memory leaks

Falls ein Programm unkontrolliert Speicher aufnimmt oder den ihm zur Verfügung gestellte nicht wieder freigibt, spricht man von einem Speicherleck. Diese sind zu verhindern da diese zu einer Systemüberlastung und Abstürzen führen.

Beispiel:

```
1 char* pCh1 = new char;
2 char* pCh2 = new char;
3
4 std::cin >> *pCh1;
5
6 pCh2 = pCh1;
7 // pCh2 points now to the Ch1
  memory cell
8 // The access to the memory
  cell of pCh2
9 // is now lost (memory leak!)
10
11 delete pCh1;
12 delete pCh2;
13 // ERROR, already with pCh1
  freed
```



11.3 Speichermanagement funktionen

Es gibt einige Funktionen für Speichermanagement. In C und C++ sind diese leicht verschieden. Generell geben diese einen Pointer zurück, welcher auf freien Speicher zeigt. **Diese müssen immer auf einen Nullpointer geprüft werden, da keine Garantie vorhanden ist, ob überhaupt Speicher vorhanden ist!** Generell sollte mit solchen Pointer immer misstrauisch gearbeitet werden und nach Abschluss immer wieder freigegeben werden.

11.3.1 C: malloc(), calloc(), free()

malloc() gibt einen Voidpointer(pointercasting nicht vergessen) zurück welche auf eine angeforderte Grösse an Bytes Speicher zeigt. Der Speicher ist **nicht** initialisiert **calloc()** macht dasselbe, initialisiert aber den Speicher auf 0. **free()** gibt den Speicherbereich wieder frei.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void *malloc(size_t size);
5 // Alokiert Speicherbereich size in Bytes
6 void *calloc(size_t size);
7 // Alokiert Speicherbereich size in Bytes und setzt diesen 0
8 free(void *ptr);
9 // Gibt Speicherbereich auf welcher ptr zeigt wieder frei
10
11 int main(){
12     int* iPtr = NULL;
13     iPtr = (int*)malloc(3*sizeof(int));
14     // Speicher fuer 3 Ints reservieren
15     // Insert Nullpointer check here!!!
16     free(iPtr);
17 } // Speicher wider freigegen
```

Falls kein Speicher allokiert werden kann, wird ein NULL-pointer zurückgegeben.

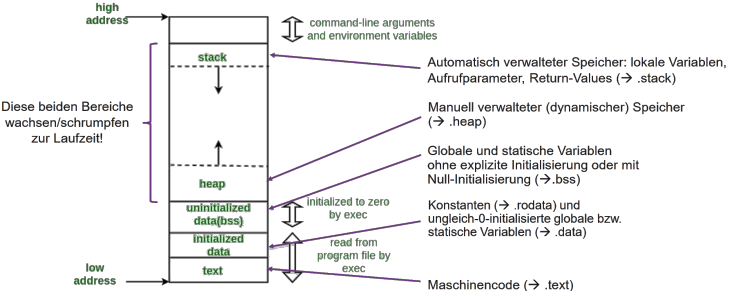
11.3.2 C++: new, delete, new[], delete[] (Operatoren)

Der **new**-Operator erstellt ein Pointer welcher auf die mitgegebene Grösse zeigt. **new[]** macht dasselbe, ausser das dieser ein Array zurückgibt. **delete** / **delete[]** gibt den Speicher wieder frei. Dieser kann auch auf den Nullpointer ausgeführt werden. Dieser ist Delete egal.

```
1 int main(){
2     int* intPtr = new int;
3     // Alokiert fuer intPtr ein Speicherbereich eines Ints
4     int* intArrayPtr = new int[10];
5     // Alokiert fuer intArrayPtr ein Speicherbereich eines
      Int Array der groesse 10
6     if(intPtr == nullptr) return -1;// Nullpointercheck
7
8     delete intPtr; // Gibt intPtr wieder frei
9     delete[] intArrayPtr; // Gibt intArrayPtr wieder frei
10    intArrayPtr = intPtr = nullptr
11 }
```

11.4 Speicheraufbau

Der Speicheraufbau bildlich dargestellt sieht ungefähr so aus:



Je nach Architektur kann es auch sein, dass die hohen und tiefen Adressen anders herum sind.

11.5 Referenzen

Una referenza è un **alias** (un nome alternativo) per una variabile o una cella di memoria già esistente. Rappresentano una versione più moderna, sicura e ristretta dei puntatori.

11.5.1 Caratteristiche Principali

- **Inizializzazione obbligatoria:** Deve essere inizializzata subito: `int& r = x;`
- **Immutabilità:** Non può essere modificata per puntare a un altro oggetto dopo la definizione.
- **Sicurezza:** Non possono essere mai non inizializzate e non hanno mai il valore `nullptr`.
- **Efficienza:** Possono risiedere nei registri della CPU e non occupano sempre memoria fisica. L'operatore `sizeof()` restituisce la dimensione del tipo a cui la referenza punta.
- **Sintassi:** Vengono utilizzate come variabili normali, senza necessità di dereferenziazione.
- **Limitazione con gli Array:** Le referenze non possono essere usate per scorrere gli array tramite aritmetica (incremento/decremento). Per questo scopo, i **puntatori** rimangono indispensabili.

11.5.2 Confronto: Value vs Reference

Caratteristica	Call by Value	Call by Reference
Copia dei dati	Sì (costoso per oggetti grandi)	No (passa l'alias)
Modifica originale	No	Sì
Sintassi	<code>void f(int a)</code>	<code>void f(int& a)</code>

11.5.3 Best Practices

L'utilizzo delle referenze riduce drasticamente i rischi di errori rispetto ai puntatori. Tuttavia, i puntatori rimangono necessari in casi specifici:

- **Gestione degli Array:** Un array "decade" naturalmente in un puntatore al suo primo elemento (*pointer decay*), rendendo i puntatori lo strumento standard per iterare o manipolare blocchi di memoria contigui.
- **Programmazione Low-level:** Necessaria per l'interazione diretta con l'hardware o allocazione dinamica manuale (`new/delete`).

```
// 1. Usa const per oggetti grandi in sola lettura
// Ottimizza le prestazioni impedendo modifiche accidentali
int foo(const BigType& b);

// 2. Passa SEMPRE struct e classi per referenza
void process(MyClass& obj);

// 3. ERRORE DA EVITARE: mai tornare referenze locali
int& func() {
```

```
int x = 10;
return x; // PERICOLO: x viene distrutta all'uscita!
} // La referenza diventerebbe "dangling" (punta al vuoto)
```

11.6 Speicherplatzbedarf von Objekten

Eine Unterklasse braucht immer mindestens so viel Speicher wie eine Oberklasse.

11.6.1 Zuweisungen

Zuweisungen von Oberklasse zu Unterklasse sind in der Regel möglich, da alle geerbten Attribute ja vorhanden sind. Umgekehrt geht das allerdings nicht, da der Speicher dafür nicht initialisiert ist.

11.6.2 Slicing Problem

- **Definizione:** Si verifica quando un oggetto di una sottoclasse viene passato per valore a una funzione che si aspetta la superclasse.
 - Viene invocato il *Copy-Constructor* della superclasse.
 - Solo la parte "base" dell'oggetto viene copiata; le estensioni della sottoclasse vengono perse (slicing).
- **Conseguenza:** All'interno della funzione, l'oggetto si comporta esclusivamente come un'istanza della classe base, perdendo ogni comportamento polimorfo.
- **Soluzione:** Utilizzare sempre il passaggio **per referenza** (`const T&`) o tramite puntatori. Questo preserva l'integrità dell'oggetto originale e permette il corretto funzionamento dei metodi `virtual`.

11.7 Type Casting

Cast:	Anwendungsbereiche	Wissenswertes	Beispiele
implicit; kein Cast	• Numerische Typen u. <code>bool</code> (ggf. mit Warnung, falls Wertebereich möglicherweise nicht ausreicht) • Objektinstanzen einer Unterklasse in einer Variablen vom Typ der Oberklasse speichern (Upcast). • Pointertypen, deren Umkonvertierung «ungefährlich» ist – die genauen Regeln sind ziemlich kompliziert. • Alles, was implizit möglich ist, unterdrückt dabei etwaige Warnungen. • Pointer- und Referenz-Typen auf Instanzen von Ober- und Unterklassen in beide Richtungen umwandeln; Upcast und Downcast (ohne Typprüfung zur Laufzeit, Auswertung ausschließlich zur Compilezeit)	Per convertire un tipo intero più grande in uno più piccolo, il compilatore semplicemente "taglia" i bit in eccesso, mantenendo solo quelli meno significativi (LSB).	1. signed char a = 1; int b = a; 2. int a = 1; char b = a; // ggf. mit Warnung 3. Bird b; Animal a = b; // Upcast: a ist ein Animal
static	• Pointer- und Referenz-Typen auf Instanzen von polymorphen Ober- und Unterklassen in beide Richtungen umwandeln; Upcast und Downcast (mit Typprüfung per RTTI zur Laufzeit)	Unterstützt ausserdem einen Upcast für Pointer- und Referenztypen auf nichtpolymorphe Klassen (normalerweise machen wir dies aber implizit oder verwenden einen <code>static_cast</code> !) Falls der <code>dynamic_cast</code> fehlschlägt , liefert er bei Pointer-Typen den Wert <code>nullptr</code> , bei Referenzen wird eine <code>std::bad_cast-Exception</code> geworfen	1. int a = 1; char b = static_cast<char>(a); 2. Bird* b = new Bird; Animal* a = b; // geht implizit! Bird* c = static_cast<Bird*>(a); /* Braucht einen down-Cast, weil nicht jedes Animal ein Bird ist! */
dynamic	• Pointer- und Referenz-Typen auf Instanzen von polymorphen Ober- und Unterklassen in beide Richtungen umwandeln; Upcast und Downcast (mit Typprüfung per RTTI zur Laufzeit)	Unterstützt ausserdem einen Upcast für Pointer- und Referenztypen auf nichtpolymorphe Klassen (normalerweise machen wir dies aber implizit oder verwenden einen <code>static_cast</code> !) Falls der <code>dynamic_cast</code> fehlschlägt , liefert er bei Pointer-Typen den Wert <code>nullptr</code> , bei Referenzen wird eine <code>std::bad_cast-Exception</code> geworfen	Animal* a = new Ant; Bird* b = dynamic_cast<Bird*>(a); /* Der Dynamic Cast wird immer zur Laufzeit ausgewertet, dh: b == nullptr */
const	• Verändert die <code>constness</code> und/oder <code>volatile-ness</code> des Ziels eines Pointer oder Referenz-Typs.		<code>const int val = 10;</code> <code>const int *ptr = &val;</code> <code>int *ptr1 = const_cast<int*>(ptr);</code>
reinterpret	• Konvertiert zwischen beliebigen Pointer-Typen sowie zwischen beliebigen Pointer und int-Typen ohne Typprüfung. • Kann auch mit Referenz-Typen umgehen.	Achtung: erzeugt platformspezifisches Verhalten	Animal* a = new Ant; uint8_t* b = reinterpret_cast<uint8_t*>(a);

Die C Syntax für Casting sollte nicht mehr verwendet werden da meistens nicht optimales Verhalten.

11.8 Templates

Templates erlauben es Typen-unabhängigen Code zu schreiben. Eine Funktion wird mit einem **Platzhalter-typ** geschrieben. Bei der Verwendung wird ein oder mehrere zu verwendende Typen mitgegeben (es ist auch möglich Konstanten mitzugeben). Es sind Klassen und Funktionstemplates möglich. Templates werden immer zur **Compilezeit** ausgewertet. Templates brauchen eine Deklaration, Definition und Instanziierungsparameter. Sie werden ausschliesslich in einem H-File deklariert. Achtung! Templates werden für jeden **unterschiedlichen** Typen der Instanziiert wird angelegt. Das kann zu hohem Speicherbedarf führen.

11.8.1 Implementation

Eine Funktion / Klasse kann als Template definiert werden indem `template<typename1 T1, typename2 T2, ...>` (mit<>) vor die Funktion / Klasse geschrieben wird. Die Typendefinitionen sind dann durch das T zu ersetzen.

```
1 // H-File
2 template<typename T>
3 void swap(T& a, T& b); // deklaration
4
5 template<typename T> // definition
6 void swap(T& a, T& b) {
7     T temp{b};
8     b = a;
9     a = temp;
10 }
```

11.8.2 Syntax

```
1 // Template-Definition fuer Funktionsdeklaration:
2 template<typename T1, typename T2, ...>
3 returntype name(type1 param1, type2 param2 , ...);
4
5 // Template-Definition fuer Funktionsdefinition:
6 template<typename T1, typename T2, ...>
7 returntype name(type1 param1, type2 param2 , ...) { ... }
8
9 // Klassentemplate:
10 // Template-Definition fuer Klassendeklaration:
11 template<typename T1, typename T2, ...>
12 class Classname{ ... };
13
14 // Template-Definition fuer Methode einer Klasse:
15 template<typename T1, typename T2, ...>
16 returntype Class-name<T1, T2, ...>::method-name(type1 param1,
17     type2 param2 , ... ) { ... };
```

11.8.3 Instanziierung

Klassentemplates (<> sind nicht fakultativ):
Templatename<type1, type2, ...>Instanzname

Funktionstemplates (<> sind nicht fakultativ):
Templatename<type1, type2, ...>(param1, ...);

Falls bei der Instanziierung des Funktionstemplates der Typ bereits klar ist, können diese weggelassen werden.

12 Exceptions

Exceptions werden zur Behandlung von Fehlern verwendet. Eine Exception ist eine Instanz eines beliebigen Datentyps, welche Fehlerinformation transportiert. Eine Exception wird bei Fehlerauftritt geworfen (throw), und zur Behandlung gefangen (catch).

```
1 try {
2     // dangerous Code with throw
3 } catch (someType someName) {
4     // some error handling.
5 }
6 // normal execution continues here
```

Diese Syntax ist so in der Praxis wenig aufzufinden da das Fehler werfen meist in einer Unterfunktion gemacht wird. Eine geworfene Exception wird dann mit code um die Instanziierung gehandhabt. Dies erlaubt es Fehler von Unterfunktionen je nach Situation anders zu handhaben:

```
1 #include <iostream>
2
3 int divide(int dividend, int divisor) {
4     if (divisor == 0) throw "Division by zero!";
5     return dividend/divisor;
6 }
7 int foo(int a, int b) {
8     std::cout << "Entering foo()" << std::endl;
9     int result = divide(a, b); // can throw exception
10    std::cout << "Leaving foo()" << std::endl;
11    return result;
12 }
13
14 int main() {
15     try {
16         std::cout << foo(7,0) << std::endl;
17     } catch (const char* e) {
18         std::clog << "Not optimal but ok, " << e << std::endl;
19     } // just writes the Error to the log
20
21     // continue Program Function
22    std::clog << "Continue Programm" << std::endl;
23
24    try {
25        std::cout << foo(12,0) << std::endl;
26    } catch (const char* e) {
27        std::cerr << "Critical error: " << e << std::endl;
28        std::cerr << "Program terminating" << std::endl;
29        exit(-1);
30    }
31    std::clog << "finish Programm" << std::endl;
32    // continue Program Function
33 }
```

- Cout:
- Entering foo()
 - Not optimal but ok, Division by zero!
 - Continue Programm
 - Entering foo()
 - Critical error: Division by zero!
 - Program terminating

12.1 Mehrere Catch Blöcke

Unterschiedliche Exceptions können mit mehreren seriellen Catch Blöcken erreicht werden. Wenn der letzte Catch block mit ... aufgerufen wird, werden alle Typen Exceptions gehandelt.

```
1 int main() {
2     int x;
3     std::cin >> x;
4     try {
5         a(x); // can throw various types of exceptions
6     } catch (const char* e) { // handles only Type const char*
7         std::clog << e << std::endl;
8     } catch (int i) { // handles only Type int
9         std::cerr << i << std::endl;
```

```
10     } catch (...) { // handles whatever...
11         std::cerr << "other exception ocured" << std::endl;
12     }
13     return 0;
```

12.2 Fehlender catch block

Falls kein Passender catch Block gefunden werden kann wird die Funktion `std::terminate()` aufgerufen. Diese ruft den registrierten `std::terminate_handler` (Funktionspointer). Defaultmässig ist `std::abort()` hinterlegt welche das Programm beendet ohne Destruktoren zu rufen (crash). Falls dieses Verhalten angepasst werden soll, kann man das mit `set_terminate(std::terminate_handler f)`.

12.3 leerer Catch block

straight to hell

12.4 noexcept

Um zu signalisieren, ob eine Funktion eine Exception auslösen kann oder nicht, kann der specifier **noexcept** verwendet werden. Dafür ist aber **der Entwickler zuständig**. Der Compiler kann **nicht** prüfen, ob die Funktion tatsächlich eine Exception wirft oder nicht.

12.5 <stdexcept>

stdexcept ist dazu da um einen einfachen Austausch von Fehlerinformationen vorzunehmen. Dafür sind diverse standardisierte Fehlertypen als Klassen definiert. Z.b.: `std::logic_error` `std::out_of_range` `std::overflow_error` Für Details siehe : <https://en.cppreference.com/w/cpp/header/stdexcept>

13 std:vector

std:vector ist ein Klassentemplate welches die Kapazität selbständig bei Bedarf erweitert.

```
1 #include <vector>
2 std::vector<dt> v; // Template instanzieren mit <datentyp> (dt)
3 v.push_back("Value");// Value anfüegen
4 v.erase("iterator");// Elemente loeschen(Iterator=fancy
5     Pointer)
6 v.begin();// returned ein Pointer auf das erste element
7 v.end();// returnd ein Pointer HINTER das letzte Element
8
9 for (const dt s : v) { // Iteriert ueber Elemente vom Vector
10     doSomething(s);} // do something mit element
11
12 for (std::vector<dt>::iterator it=v.begin();it!=v.end();it++)
13     {
14         doSomething(s);} // do something mit element, altes for
```

14 Makefiles

Makefiles sollten generell das Umsetzen des Codes in Maschinencode vereinfachen. Es ist eine Skriptingsprache welche grob nach dem Muster `Erzeugnis/target : Abhängigkeiten/Dependency` folgt. Dann folgt indentiert der Befehl, um dieses Erzeugnis zu erzeugen. Wenn ein Erzeugnis keine Datei zurückgibt, muss dieser als `.PHONY` markiert werden. Das verhindert, dass eine Datei mit demselben Namen das Ausführen verhindert. Am besten sieht man das an einem Beispiel:

```
1 all : stuff # erzeugt die Definition fuer make
2     all
```



```

2
3 project : main.o lib.o      # Projekt linken
4   clang++ -Wall -o vector Vector3D.o main.o
5
6 lib.o : lib.cpp             # lib.cpp compilieren
7   clang++ -Wall -c lib.cpp
8
9 main.o : main.cpp           # main.cpp compilieren
10  clang++ -Wall -c main.cpp
11
12 clean :                     # Projekt cleanup
13   rm -f project.exe lib.o main.o
14
15 .PHONY : clean all          # Mariert die Targets "clean" und
   "all" als "PHONY" -> Lesbarkeit

```

Der Befehl `make all` baut nun das Projekt.
 Sind die `o` Dateien noch aktuell werden diese nicht erneut kompiliert. Das Projektverzeichnis kann einfach durch `make clean` aufgeräumt werden.

14.1 Platzhalter

In einem Makefile können auch Platzhalter verwendet werden:

- `$ @` Dateiname des Targets
- `$ <` Dateiname der ersten Dependency des aktuellen Targets
- `$ ^` Dateiname aller Dependencies des aktuellen Targets durch Leerzeichen getrennt

Mit diesen Mitteln kann ein Professionelleres Makefile erstellt werden, welches relativ universell eingesetzt werden kann:

```

1 CXX = clang++               # Verwendeter Compiler
2 CXXFLAGS = -Wall -c         # compilerflags
3 LDFLAGS = -o                # loader flags
4 BIN = main                  # binary output / Name des .exe
   file
5 OBJS = drink.o drinktest.o # objectfiles
6
7 all: $(BIN)                 # Make all befehl
8
9 %.o: %.cpp                  # cpp Files zu o dateien compilieren
10   $(CXX) $(CXXFLAGS) $<
11
12 $(BIN): $(OBJS)             # Linken zu exe file
13   $(CXX) $(LDFLAGS) $@ $^
14
15 run: $(BIN)                 # Projekt ausfuehren
16   ./$(BIN)
17
18 clean:                      # o. Dateien entfernen
19   rm -f $(OBJS) $(BIN)
20
21 .PHONY: clean all           # Markiert clean & all als PHONY

```

15 Anhang

15.1 Styleguide

- **Variabili, Costanti**

- mit Kleinbuchstaben beginnen
- erster Buchstaben von zusammengesetzten Wörtern ist gross (mixed case)
- keine Underscores

Esempi : counter, maxSpeed

- **Funktionen**

- mit Kleinbuchstaben beginnen
- erster Buchstaben von zusammengesetzten Wörtern ist gross (mixed case)
- Namen beschreiben Tätigkeiten
- keine Underscores

Esempi : getCount(), init(), setMaxSpeed()

- **Klassen, Strutturen, Enums**

- mit Grossbuchstaben beginnen
- erster Buchstaben von zusammengesetzten Wörtern ist gross (mixed case)
- keine Underscores
- Namen beschreiben Dinge

Esempi : MotorController, Queue, Color

15.2 Code esempi

15.2.1 String formatting

Utilizza gli stream di output per costruire stringhe in memoria. È sicuro perché gestisce la memoria dinamicamente.

```
1 #include <sstream>
2 #include <iomanip>
3
4 std::string formatTime(int h, int m) {
5     std::ostringstream buf;
6     buf << std::setfill('0') << std::setw(2) << h
7         << ":" << std::setw(2) << m;
8     return buf.str();
9 }
```

Rappresenta lo standard moderno. Combina la sintassi concisa di printf con la sicurezza degli stream.

```
1 #include <format>
2
3 std::string formatTime(int h, int m) {
4     // :02 significa 2 cifre con riempimento zero
5     return std::format("{:02}:{:02}", h, m);
6 }
```

15.2.2 Ctor un dtor

```
1 class TString {
2 public:
3     TString();                // 1: Default Ctor
4     TString(const TString& s); // 2: Copy Ctor
5     TString(const char* p);    // 3: Ctor da stringa
6     explicit TString(int i);   // 4: Ctor esplicito da
7     int                       // ...
8 };
9
```

```
10 TString s1 = "Hoi";         // 3
11 TString s2;                 // 1
12 TString s21 = s1;           // 2
13 s2 = s1;                    // Nessun Ctor (Assignment operator)
14 TString& s3 = s21;          // Nessun Ctor (Riferimento)
15 TString s4[2];              // 1, 1 (Array di 2 elementi)
16 TString* p1 = &s21;         // Nessun Ctor (Puntatore)
17 TString* p2 = new TString{"Hallo"}; // 3 (Allocazione su heap)
18 TString* p3 = new TString[3]; // 1, 1, 1 (Array di 3 su
19     heap)
20
21 TString s5[3] = {"Hoi", TString{}, s2}; // 3, 1, 2
22
23 // Esempio per il Ctor 4 (explicit)
24 TString s13 = TString{13};      // 4
```

15.3 Ritorno per Riferimento e Method Chaining

- **Meccanismo:** Una funzione restituisce un riferimento all'oggetto stesso utilizzando `return *this;`. Il tipo di ritorno deve essere una referenza (es. `Complex&`).
- **Kaskadierung (Cascading):** Permette di chiamare più metodi in sequenza sulla stessa istanza in un'unica riga: `obj.funz1().funz2();`.
Tramite ritorno *by-value* `funz2` verrebbe eseguita su un oggetto *tmp*
- **Efficienza:** A differenza del ritorno per valore, non viene creata alcuna copia dei dati (nessun oggetto temporaneo *tmp*), risparmiando risorse
- **Persistenza:** Le modifiche effettuate nella catena agiscono direttamente sull'oggetto originale e non su una sua copia destinata alla distruzione.

```
1 // .h
2 class Complex
3 {
4     public:
5     Complex(const Complex& c); // Copy constructor
6     Complex& add(const Complex& c); // adds c to the current
7     object
8 };
9 // complex.cpp
10 Complex& Complex::add(const Complex& c) {
11     this->re += c.re;
12     this->im += c.im;
13     return *this;
14 }
15
16 // main.cpp
17 Complex c3 = c1.add(c0).add(c2);
18 // c1 viene aggiornato correttamente
19 // c3 viene creato tramite copy ctor
```